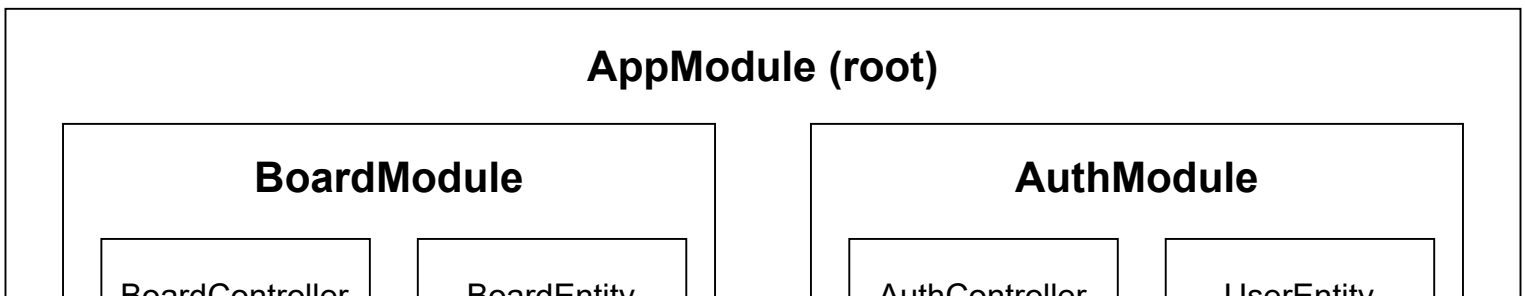
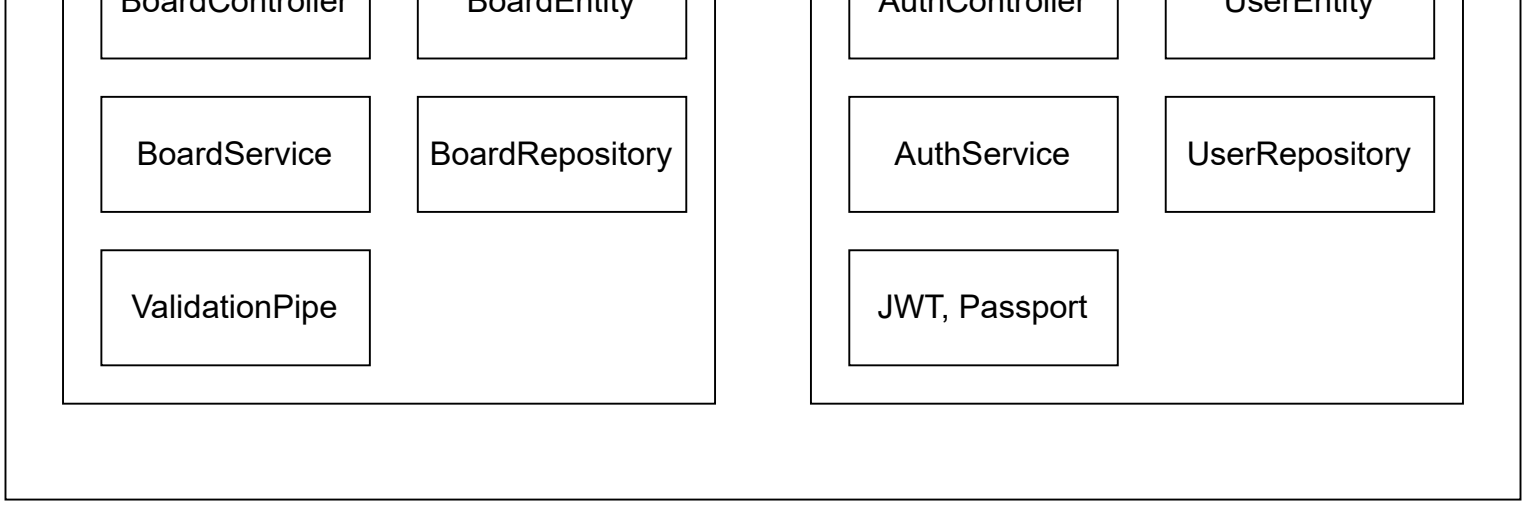
게시물 CRUD 애플리케이션 소개



이 앱을 만들 때 Nest JS에서 사용할 것들 (앱 구조)

게시글을 만드는 것이기에 게시글에 관한 모듈과 그 게시글을 만드는 사람에 대한 인증 모듈이 필요합니다. 그리고 각 모듈을 구성하는 Controller, Service, Repository 등이 있는데 이러한 용어는 Express를 사용할 때도 들어보았을 수 있지만 NestJS에서는 어떠한 용도로 사용되는지




BoardController
BoardEntity
BoardService
BoardRepository
ValidationPipe
AuthController
UserEntity
AuthService
UserRepository
JWT, Passport

프로젝트 시작하기

프로젝트를 시작 할 폴더 생성

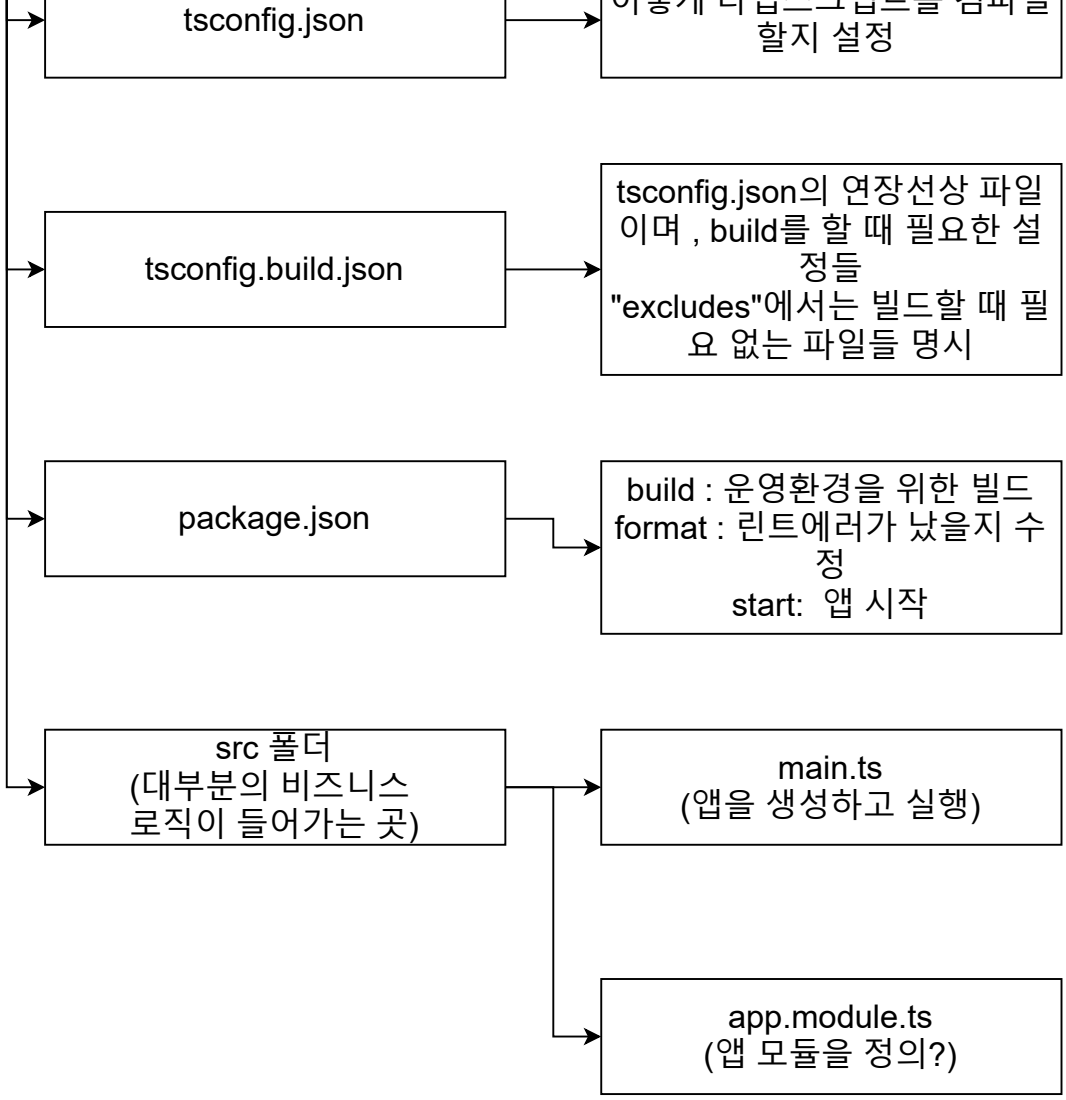
폴더 안에서 nest 기본 구조 생성 → npm i -g @nestjs/cli

nest new ./

앱 실행 → npm start:dev

Nest JS 기본 구조 설명

- eslintrc.js → 개발자들이 특정한 규칙을 가지고 코드를 깔끔하게 짤수있게 도와주는 라이브러리 타입스크립트를 쓰는 가이드 라인 제시, 문법에 오류가 나면 알려주는 역할 등등
- prettierrc → 주로 코드 형식을 맞추는데 사용합니다. 작은따옴표(')를 사용할지 큰 따옴표(")를 사용할지, Indent 값을 2로 줄지 4로 줄지 등등, 에러 찾는것이 아닌 코드 포맷터 역할.
- nest-cli.json → nest 프로젝트를 위해 특정한 설정을 할 수 있는 json 파일
- 어떻게 타입스크립트를 컴파일


tsconfig.json
어떻게 타입스크립트를 컴파일 할지 설정
tsconfig.build.json
tsconfig.json의 연장선상 파일이며 , build를 할 때 필요한 설정들
"excludes"에서는 빌드할 때 필요 없는 파일들 명시
package.json
build : 운영환경을 위한 빌드
format : 린트에러가 났을지 수정
start: 앱 시작
src 폴더
(대부분의 비즈니스 로직이 들어가는 곳)
main.ts
(앱을 생성하고 실행)
app.module.ts
(앱 모듈을 정의?)

기본 구조에서 살펴보는 NestJS 로직 흐름

Hello World 텍스트를 출력하는 API

ExpressJS를 쓸 때와 Nest JS를 쓸 때 코드의 차이를 비교해서 Nest JS 로직의 흐름이 어떻게 이뤄지는지 이해해보겠습니다.

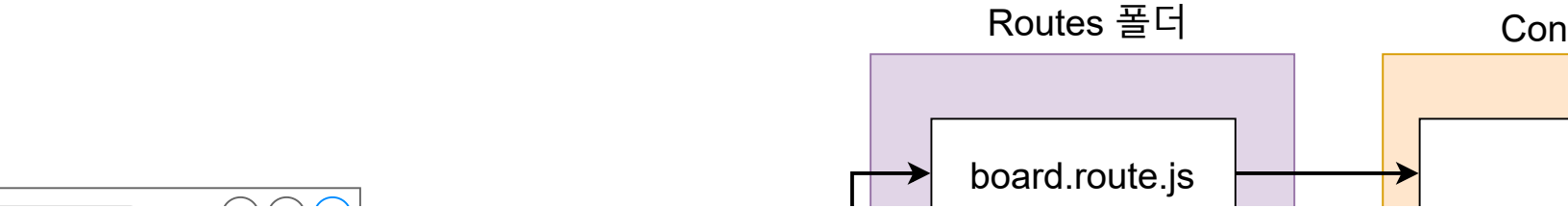


NestJS

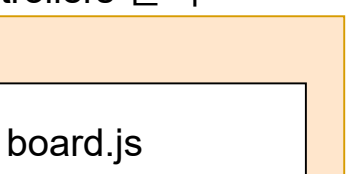
Controller → Service

```
@Get()
getHello(): string {
  return this.appService.getHello();
}
```

```
@Injectable()
export class AppService {
  getHello(): string {
    return 'Hello World!';
  }
}
```



trollers 폴더



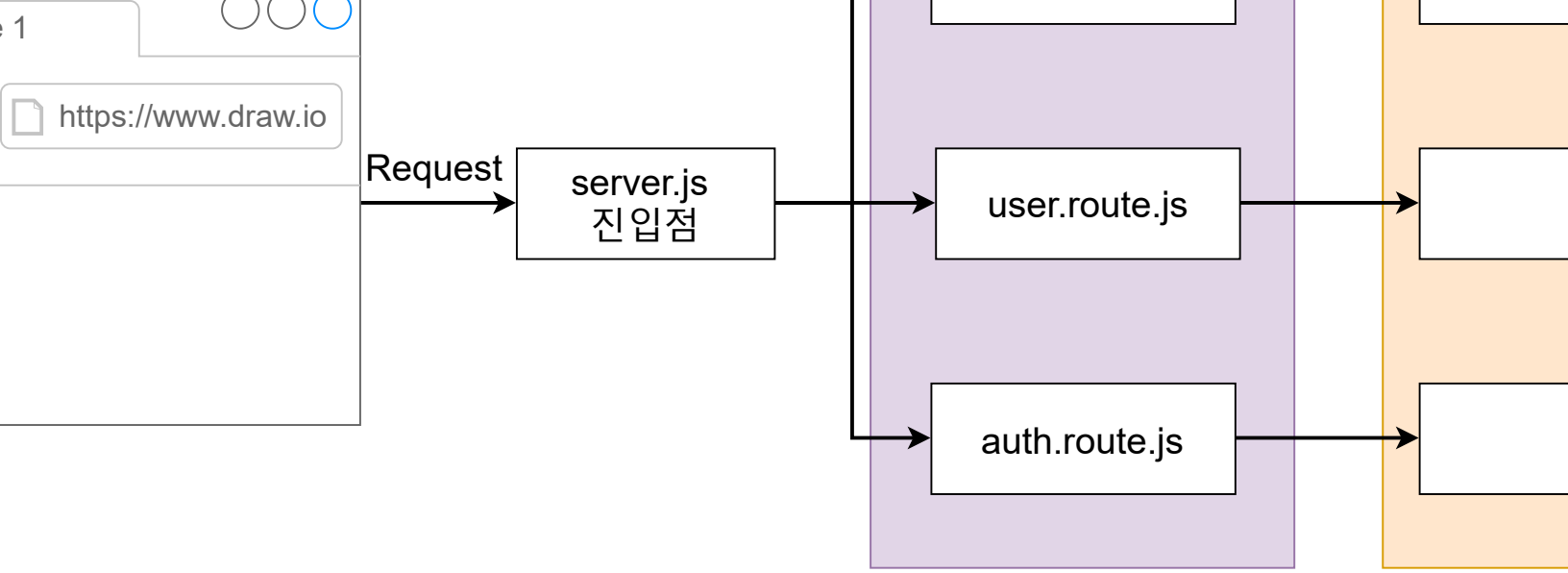


s

```
app.use('/api/board
app.use('/api/user'
app.use('/api/auth'
```

app

```
@Module({
  imports: [
    TypeOrmModu
    BoardsModul
    AuthModule,
    UserModule,
  ],
})
export class Ap
```



server.js
진입점

```
', require('./routes/board'));
, require('./routes/user'));
, require('./routes/auth'));
```

routes/user.route.js

```
router.get("/", userController.getUsers)
router.get("/:bandId", userController.getUser);
router.put("/", userController.updateUser);
router.delete("/", userController.deleteUser)
```

진입점
o.module.ts

```
ule.forRoot(typeORMConfig),
le,

pModule {}
```

board.controller.ts

```
@Get()
getAllBoards(
  @GetUser() user: User,
): Promise<Board[]> {
  this.logger.verbose(`User "${user.username} trying to get all boards
  return this.boardsService.getAllBoards(user);
}

@Get('/:id')
getBoardById(
  @Param('id', ParseIntPipe) id: number,
  @GetUser() user: User,
): Promise<Board> {
  return this.boardsService.getBoardById(id, user);
}

@Post()
@UsePipes(ValidationPipe)
createBoard(
  @Body() createBoardDto: CreateBoardDto,
  @GetUser() user: User,
): Promise<Board> {
  this.logger.verbose(`User "${user.username}" creating a new board.
  Payload: ${JSON.stringify(createBoardDto)}`)
  return this.boardsService.createBoard(createBoardDto, user);
}
```

users.js

auth.js

controllers/users.js

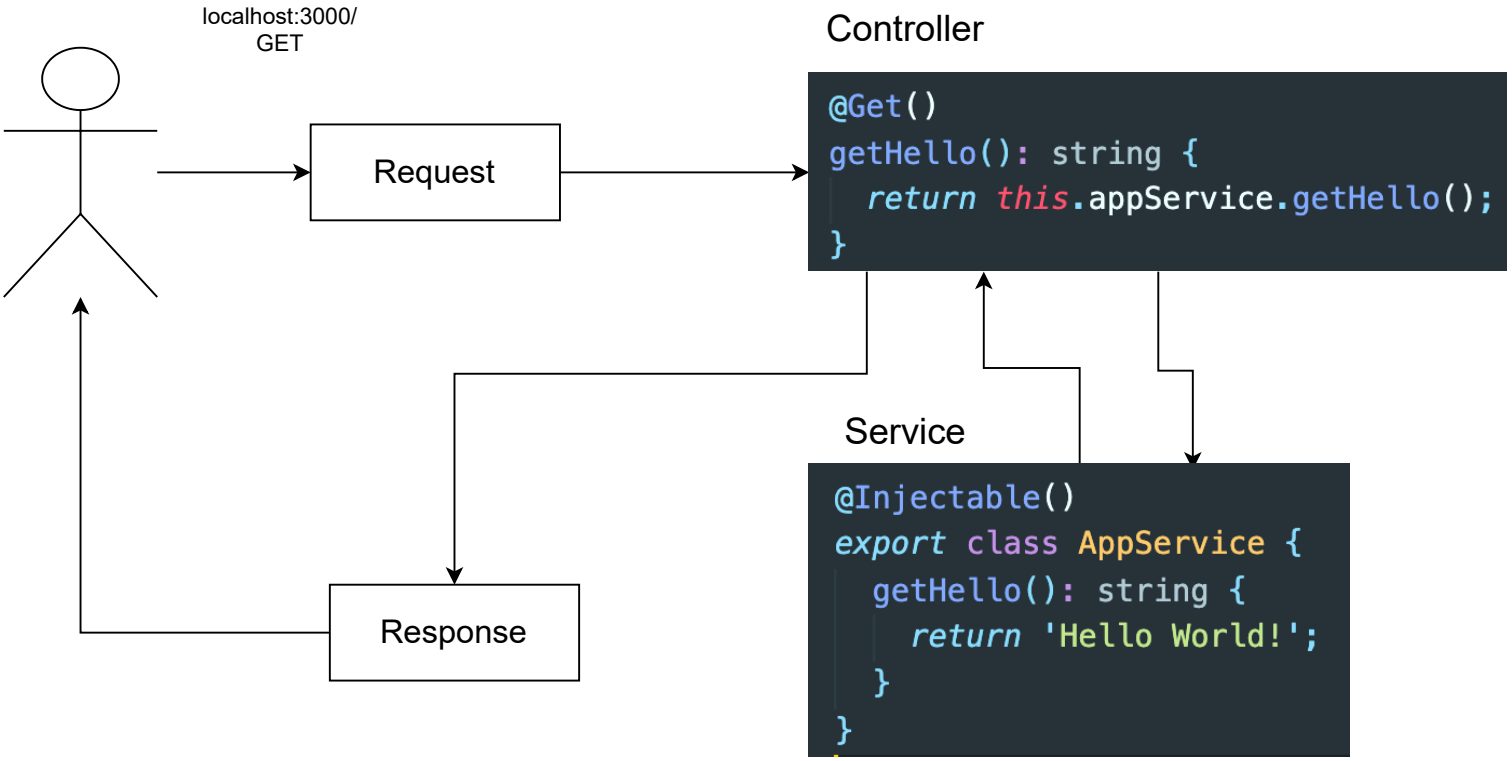
```
export const getUsers = async (req, res) => {
    let users= db.getUsers()
    return users
}
```

board.service.ts

```
async getAllBoards(
  user: User,
): Promise<Board[]> {
  const query = this.boardRepository.createQueryBuilder('board');

  query.where('board.userId = :userId', { userId: user.id })

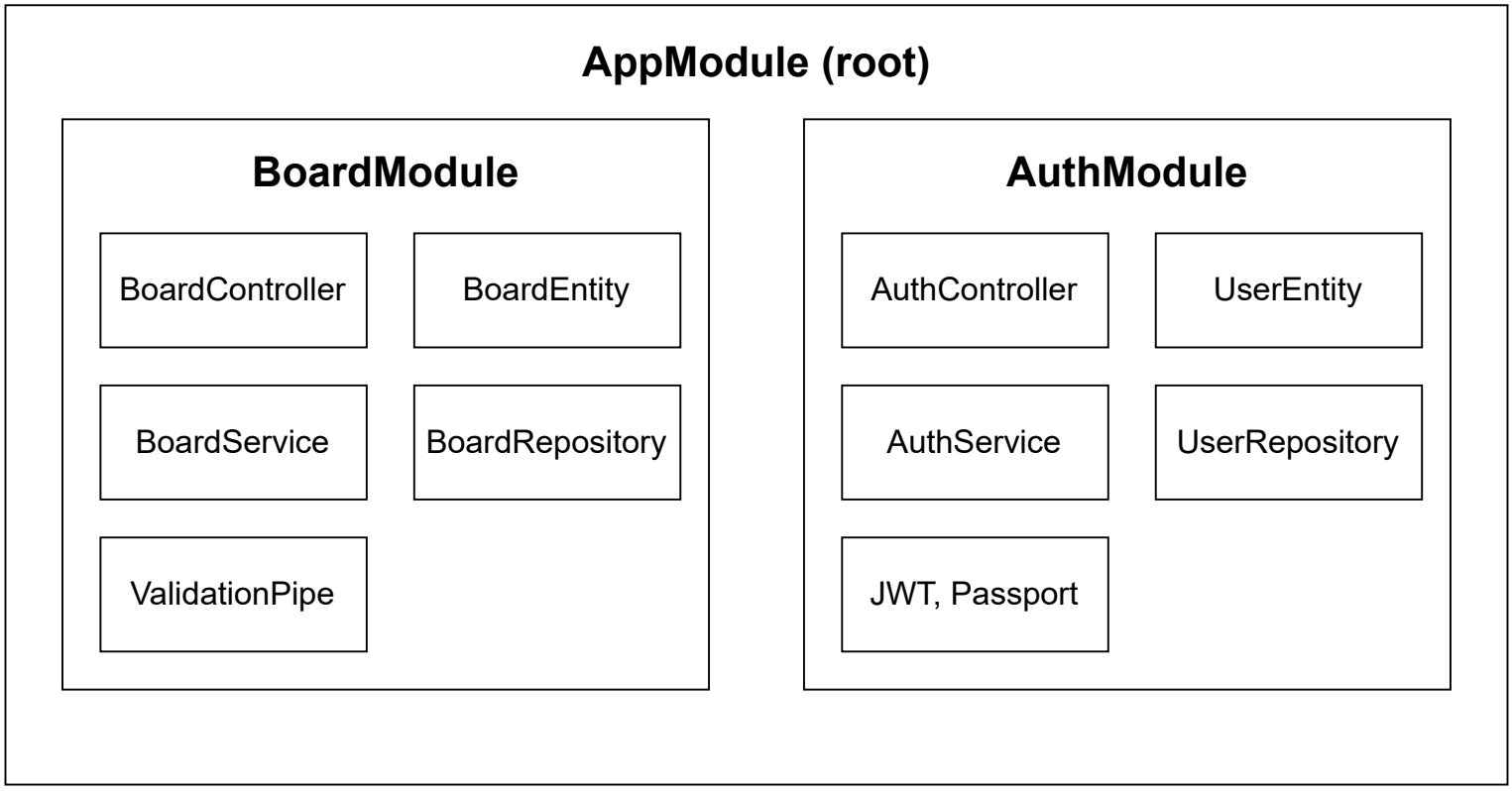
  const boards = await query.getMany();
  return boards;
}
```


localhost:3000/
GET
Request
Controller
@Get()
getHello(): string {
return this.appService.getHello();
}
Service
@Injectable()
export class AppService {
getHello(): string {
return 'Hello World!';
}
}
Response

NestJS 모듈이란 ?

Nest JS를 사용해서 만드는 앱 구조

App Module안에 BoardModule과 AuthModule이 있으면 각 모듈안에 Controller Entity Service등이 있습니다. 그래서 우선 모듈(Module)이 무엇인지 알아보겠습니다.



Nest JS 모듈이란 ?

모듈은 @Module () 데코레이터로 주석이 달린 클래스입니다. @Module () 데코레이터는 Nest가 애플리케이션 구조를 구성하는 데 사용하는 메타 데이터를 제공합니다.

각 응용 프로그램에는 하나 이상의 모듈 (루트 모듈)이 있습니다. 루트 모듈은 Nest가 사용하는 시작점입니다.



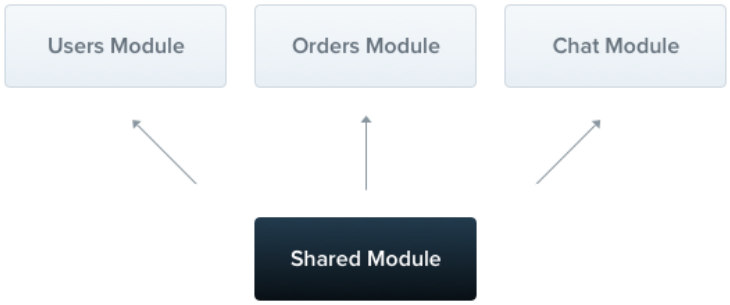


출처 nest.js

모듈은 밀접하게 관련된 기능 집합으로 구성 요소를 구성하는 효과적인 방법입니다. (기능별로 만듭니다) ex) 유저 모듈, 주문 모듈, 챗 모듈...

같은 기능에 해당하는 것들은 하나의 모듈 폴더안에 넣어서 사용합니다. (UserController, UserService, UserEntity 다 같은 기능이기에 UserModule안에 넣습니다.)

모듈은 기본적으로 싱글 톤이므로 여러 모듈간에 쉽게 공급자의 동일한 인스턴스를 공유 할 수 있습니다.



출처 nest.js

소스 코드를 이용해서 모듈을 만드는 방법은 ?

게시글을 위한 BoardModule안에 BoardController와 BoardService를 모듈안에 아래와 같이 넣어주시면 됩니다. 우선은 간단하게 이렇게 되는 것이구나만 보고 가겠습니다.

```
import { Module } from '@nestjs/common';
import { BoardsController } from './boards.controller';
import { BoardsService } from './boards.service';

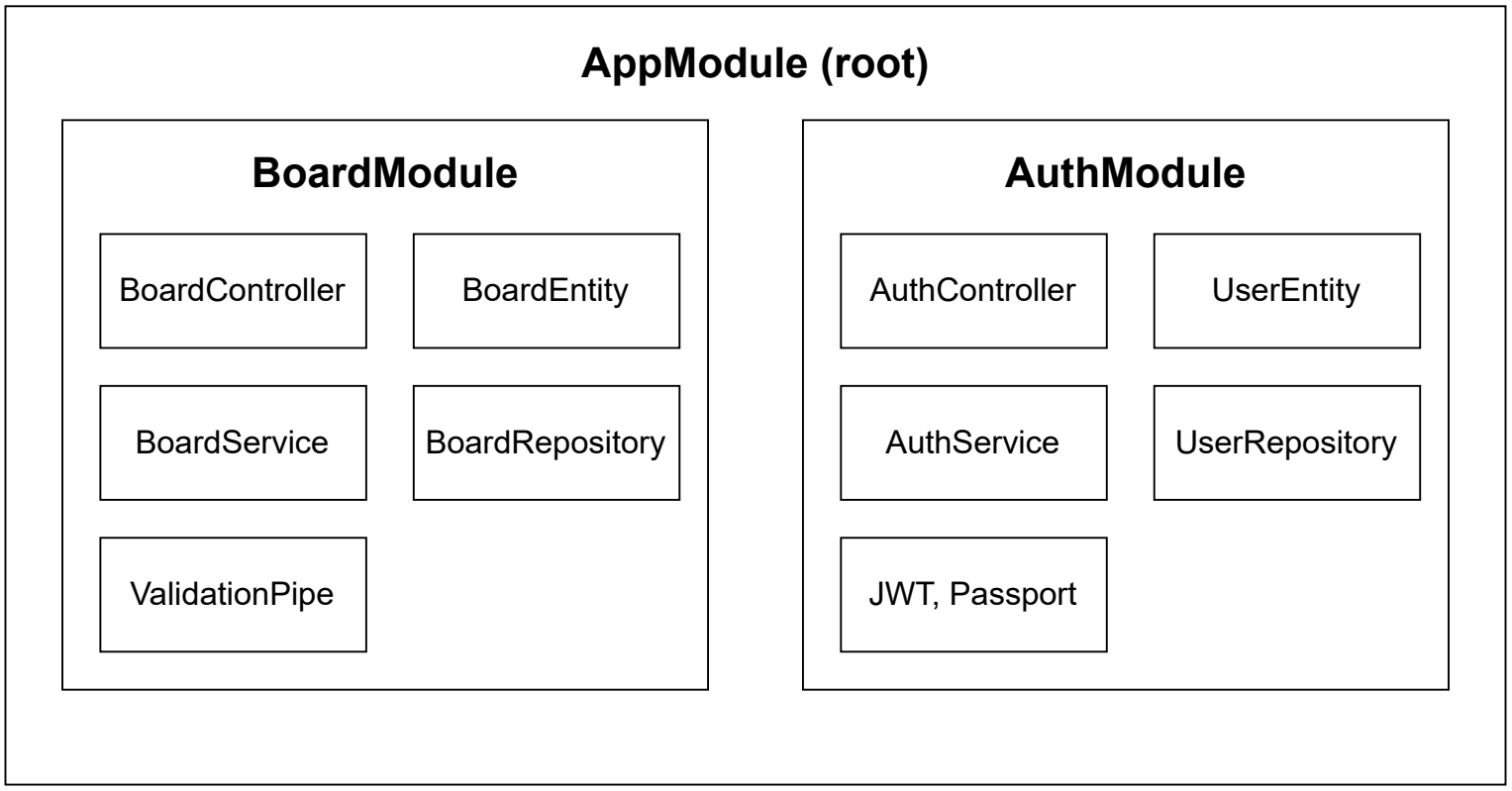
@Module({
controllers: [BoardsController],
providers: [BoardsService]
```

```
})
export class BoardsModule {}
```

Board Module 생성하기

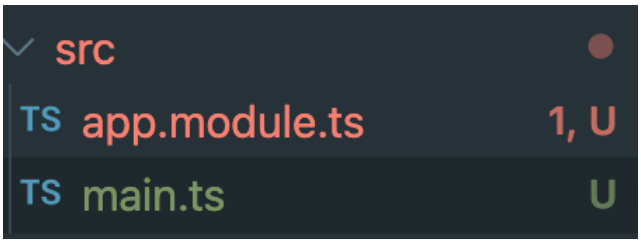
Board Module 생성하기

아래와 같은 어플을 만들기 위해서는 Root 모듈 안에 두개의 모듈이 필요합니다.
그중에서 먼저는 BoardModule을 생성해보겠습니다.



우선 처음 부터 만들기 위해서 파일 정리...

처음 부터 하나씩 만들기 위해서 root 모듈인 app.module.ts 파일과 main.ts 파일만 다 지워주겠습니다.



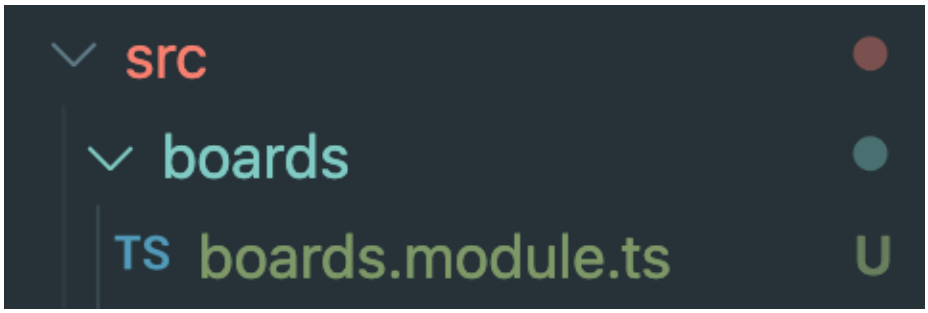

```ts
import { Module } from '@nestjs/common';

@Module({
  imports: []
})
export class AppModule {}
```

모듈 생성하기

Board 모듈 생성 명령어 → nest g module boards

nest: using nestcli
g: generate
module: schematic that i want to create
boards: name of the schematic



방금 생성한 boardModule을 사용하기 위해서는...

루트 모듈인 app.module.ts에 등록해주어야 합니다. 이건 board 모듈을 생성할 때 자동으로 등록이 됩니다.

```
import { Module } from '@nestjs/common';
import { BoardsModule } from './boards/boards.module';

@Module({
  imports: [BoardsModule]
})
export class AppModule {}
```

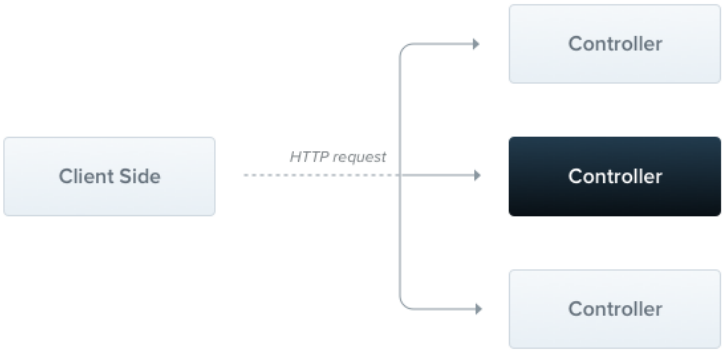
잘 실행되는지 확인하기

npm run start:dev 명령어로 실행하기

NestJS Controllers 이란?

Controller 란 ?

컨트롤러는 들어오는 요청을 처리하고 클라이언트에 응답을 반환합니다.



출처 nest.js

콘트롤러는 @Controller 데코레이터로 클래스를 데코레이션하여 정의 됩니다.

```
@Controller('/boards')
export class BoardsController {

}
```

데코레이터는 인자를 Controller에 의해서 처리되는 "경로"로 받습니다.

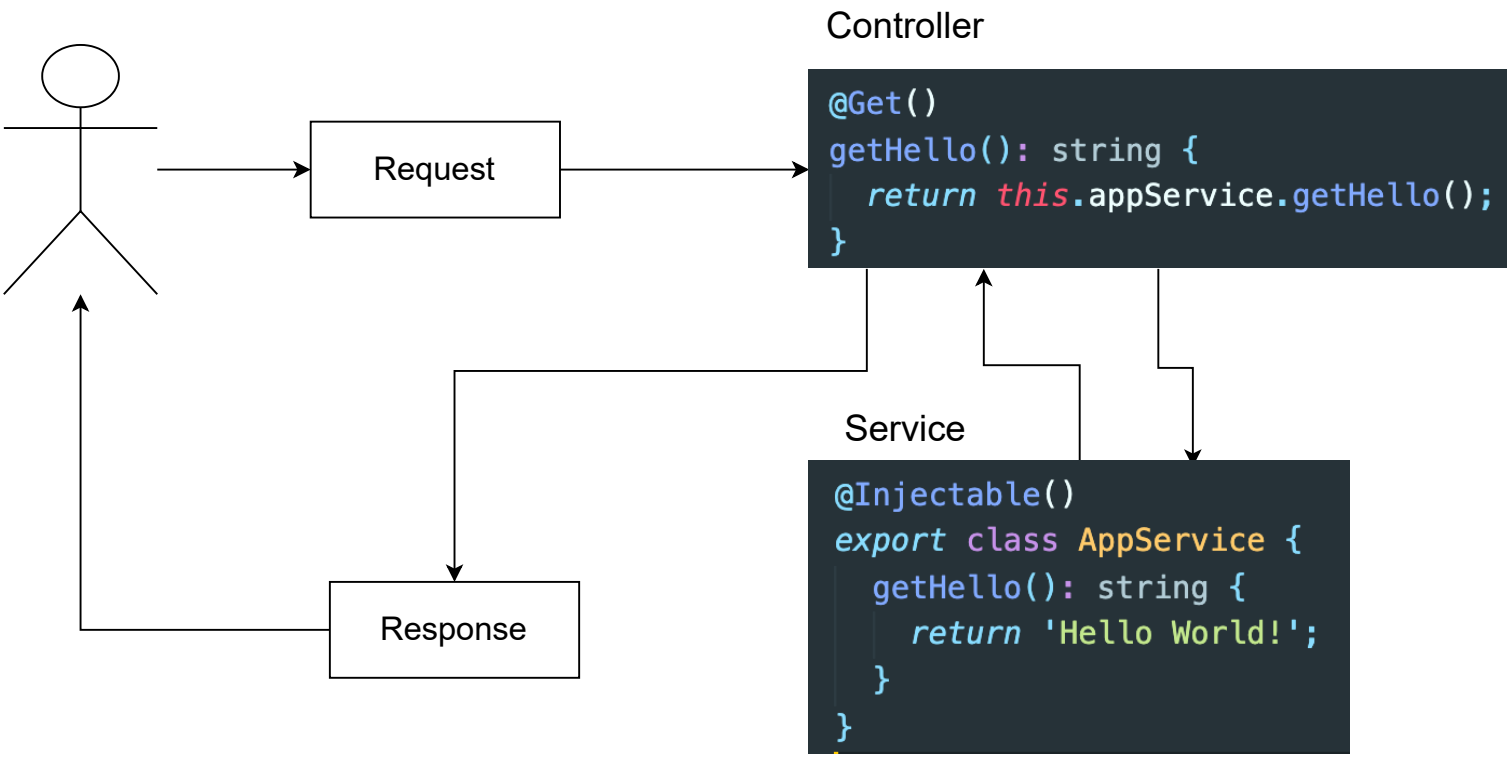
Handler 란 ?

핸들러는 @Get, @Post, @Delete 등과 같은 데코레이터로 장식 된 컨트롤러 클래스 내의 단순한 메서드입니다.

```ts
@Controller('/boards')
export class BoardsController {

    @Get()
    getBoards(): string {
        return 'This action returns all boards';
    }

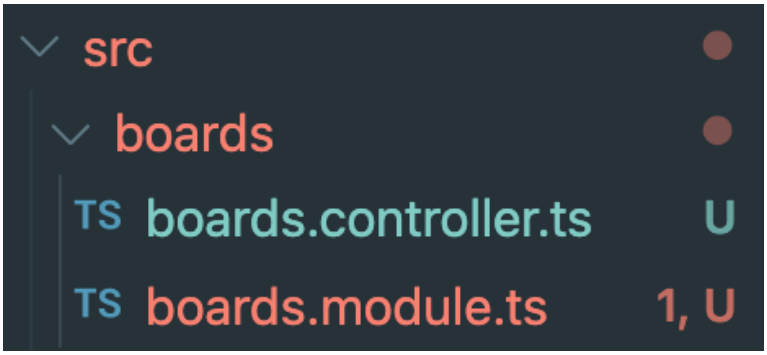
}
```



Boards Controller 생성하기

boards Controllers 생성하기 → nest g controller boards --no-spec

nest: using nestcli
g: generate
controller: controller schematic
boards: name of the schematic
--no-spec: 테스트를 위한 소스 코드 생성 x

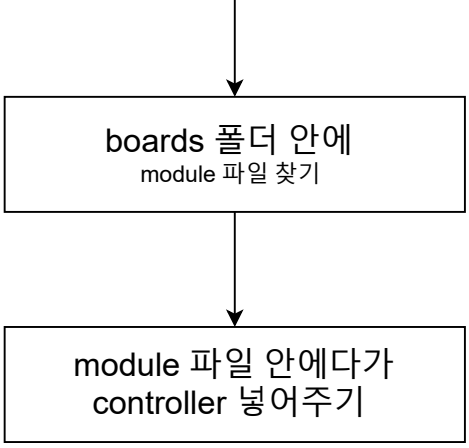



```
import { Controller } from '@nestjs/common';

@Controller('boards')
export class BoardsController {}
```

CLI로 명령어 입력 시 Controller 만드는 순

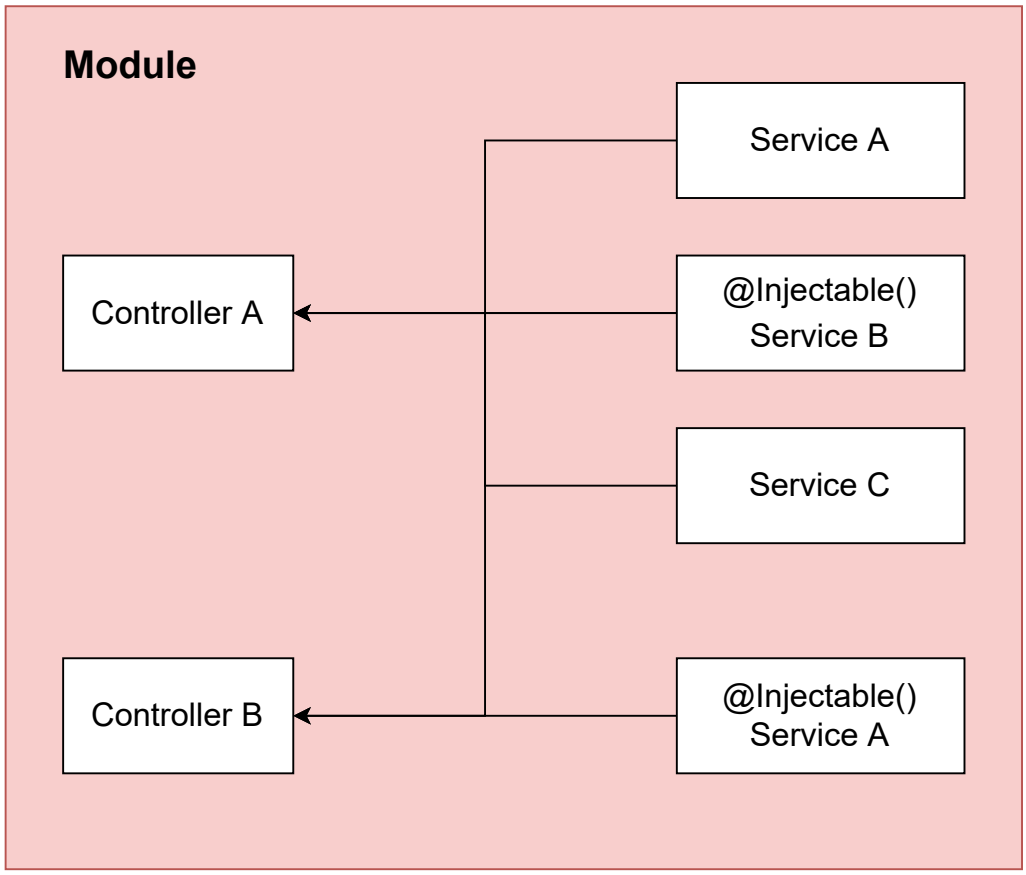
cli 는 먼저 boards 폴더 찾기 → boards 폴더 안에 controller 파일 생성


boards 폴더 안에
module 파일 찾기
module 파일 안에다가
controller 넣어주기

NestJS Providers, Service 란?

Providers 란 ?

프로바이더는 Nest의 기본 개념입니다. 대부분의 기본 Nest 클래스는 서비스, 리포지토리, 팩토리, 헬퍼등 프로바이더로 취급될 수 있습니다. 프로바이더의 주요 아이디어는 종속성으로 **주입**할 수 있다는 것입니다. 즉, 객체는 서로 다양한 관계를 만들 수 있으며 객체의 인스턴스를 "연결"하는 기능은 대부분 Nest 런타임 시스템에 위임될 수 있습니다.



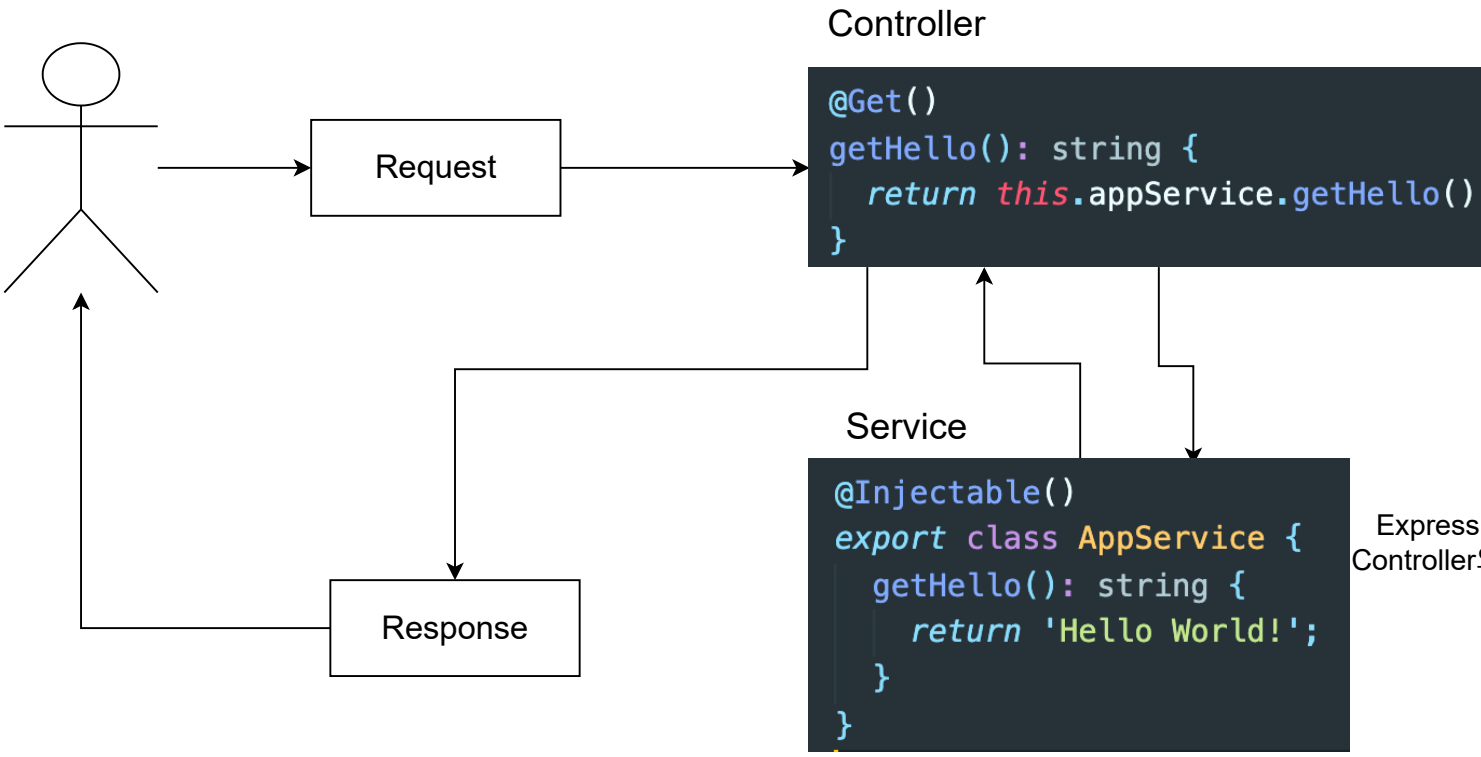
Service 란 ?

서비스는 소프트웨어 개발내의 공통 개념이며, NestJS, Javascript에서만 쓰이는 개념이 아닙니다.

@Injectable 데코레이터로 감싸져서 모듈에 제공되며, 이 서비스 인스턴스는 애플리케이션 전체에서 사용 될 수 있다.

서비스는 컨트롤러에서 데이터의 유효성 체크를 하거나 데이터베이스에 아이템을 생

성하는 등의 작업을 하는 부분을 처리합니다.



Service를 Controller에서 이용할 수 있는 방법(Dependency Injection)

위에 보면 Contoller에서 this.appService.getHello(); 이런식으로 Service에 정의해놓은 메소드를 Controller에서 가져와서 쓰는 걸 볼 수 있습니다.

```
import { Controller, Get, Post, Body, Param, Delete, Pat
import { BoardsService } from './boards.service';
import { Board, BoardStatus } from './boards.model';
import { CreateBoardDto } from './dto/create-board.dto';
import { GetBoardsFilterDto } from './dto/get-boards-fil

@Controller('boards')
export class BoardsController {
  constructor(private boardsService: BoardsService) {}

  @Get('/:id')
  getBoardById(@Param('id') id: string): Board {
    return this.boardsService.getBoardById(id);
  }
```

위에 보면 BoardsService를 Contructor 클래스에서 가져오고(Injected) 있습니다. 그런 후에 Private 문법을 사용하고 있습니다. 이렇게 해서 boardsService를 정의해서 Controller안에서 사용할수 있게 만들었습니다. 이렇게 할 수 있는 이유는 타입스크립트의 기능을 이용해서 종속성을 타입으로 해결할수 있기때문입니다.

;

에서
와 같음

Provider 등록하기

Provider를 사용하기 위해서는 이것을 Nest에 등록해줘야지 사용할 수가 있습니다.

등록하기 위해서는 module 파일에서 할 수 있습니다. module 파일에 providers항목 안에

해당 모듈에서 사용하고자 하는 Provider 를 넣어주시면 됩니다.

```
import { Module } from '@nestjs/common';
import { BoardsController } from './boards.controller';
import { BoardsService } from './boards.service';

@Module({
  controllers: [BoardsController],
  providers: [BoardsService]
})
export class BoardsModule { }
```

Boards Service 만들기

Board Service 만들기

앞에서 Service에 대해서 알아봤으니 이제 Service를 직접 만들어 보겠습니다.

Service 안에서는 데이터베이스 관련된 로직을 처리하겠습니다. 데이터베이스에서 데이터를 가져오거나 데이터베이스안에 게시판 생성할때 그 생성한 게시판 정보를 넣어주는 등의 로직을 처리하겠습니다.

board Service 생성하기

nest g service boards --no-spec

nest: using nestcli
g: generate
controller: service schematic
boards: name of the schematic
--no-spec: 테스트를 위한 소스 코드 생성 x

```
jaewon@Jaewonui-MacBookPro nest-start % nest g service boards --no-spec
CREATE src/boards/boards.service.ts (90 bytes)
UPDATE src/boards/boards.module.ts (254 bytes)
jaewon@Jaewonui-MacBookPro nest-start %
```

```
import { Injectable } from '@nestjs/common';

@Injectable()
export class BoardsService {}
```

Board Service

CLI를 이용해서 Service를 생성하면 이렇게 boards.service.ts 파일이 생성됩니다.

이 생성된 파일 안에는 Injectable 데코레이터가 있으며 이 NestJS는 이것을 이용해서 다른 컴포넌트에서 이 서비스를 사용 할 수있게(Injectable)만들어줍니다.

그리고 CLI로 Service 생성시에는 module에도 자동으로 Service가 추가됩니다.

```ts
import { Module } from '@nestjs/common';
import { BoardsController } from './boards.controller';
import { BoardsService } from './boards.service';

@Module({
  controllers: [BoardsController],
  providers: [BoardsService]
})
export class BoardsModule {}
```

Board Service를 Board Controller에서 이용할 수 있게 해주기 (Dependency Injection)

Nest JS에서 Dependency Injection은 클래스의 Constructor안에서 이루어 집니다.

```ts
import { Controller } from '@nestjs/common';
import { BoardsService } from './boards.service';

@Controller('boards')
export class BoardsController {
  constructor(private boardsService: BoardsService) {}
}
```

위 소스 코드가 만들어진 순서

위와 같은 코드는 Typescript의 도움을 받아서 가능하게 됩니다. 자바스크립트에서는 private같은 접근 제한자(Access modifier)를 사용할수 없지만 Typescript에서

```ts
@Controller('boards')
export class BoardsController {
  3 boardsService: BoardsService;

  1 constructor(boardsService: BoardsService) {
      2 this.boardsService = boardsService;
```

```
  }

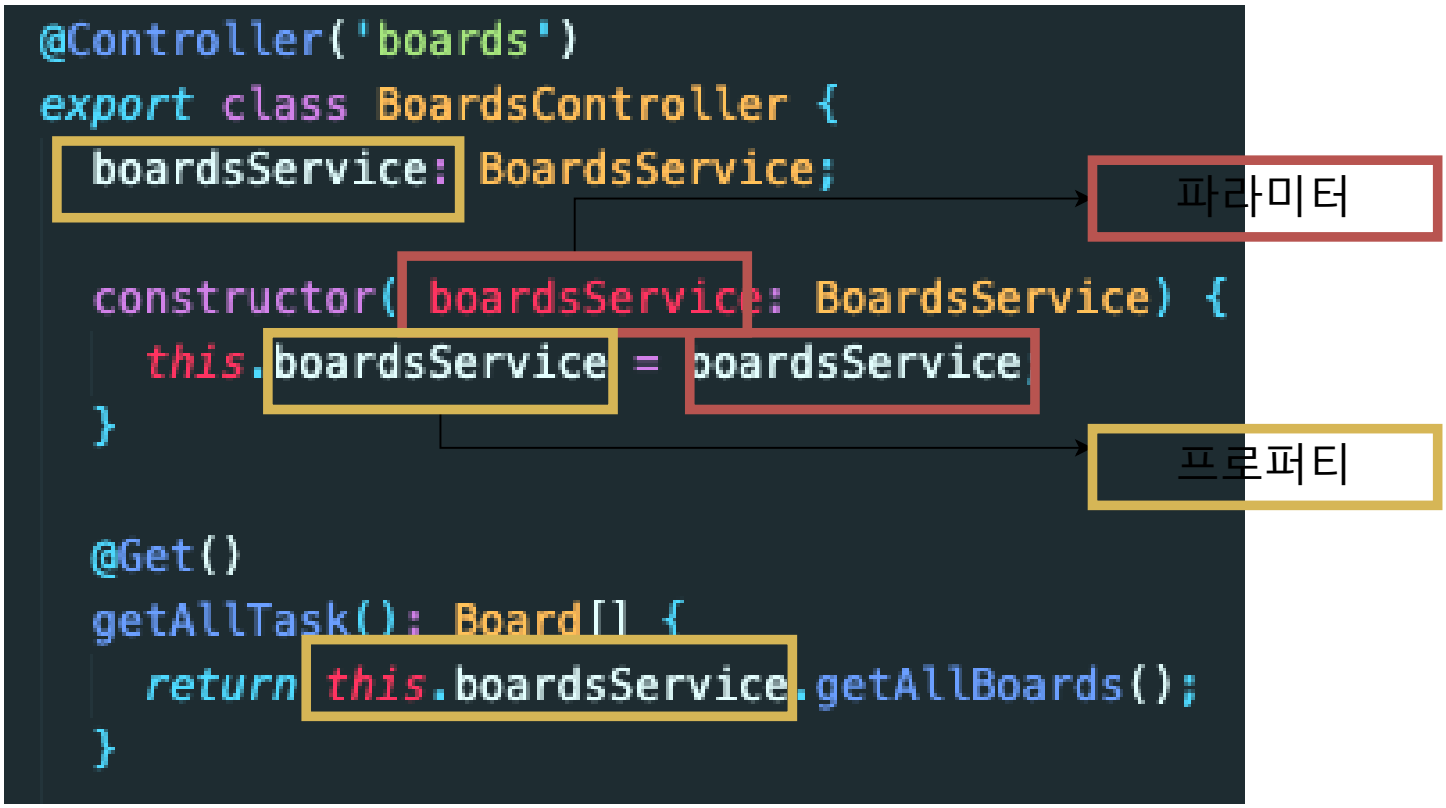
}
```

1. boardsService 파라미터에 BoardsService 객체를 타입으로 지정해줍니다.

2. 이 boardsService 파라미터를 BoardsController 클래스 안에서 사용하기 위해서 this.boardsService 프로퍼티에 boardsService 파라미터를 할당해줍니다.

3. 하지만 타입스크립트에서는 선언한 값만 객체의 프로퍼티로 사용가능하기 때문에
위에 boardsService: BoardsService로 선언해줍니다.

4. 이렇게 갖게된 boardsService 프로퍼티를 이용해서 BoardsController 클래스 안에서 활용을 할수가 있습니다.



접근 제한자를 이용해서 소스 간단하게 하기

접근 제한자(public, protected, private)을 생성자(constructor) 파라미터에 선언하면 접근 제한자가 사용된 생성자 파라미터는 암묵적으로 클래스 프로퍼티로 선언됩니다.



```
export class BoardsController {
  constructor(private boardsService: BoardsService) {}

  getAllTask() {
      this.boardsService.
  }

}
```

Private을 사용하면 ?

private이 선언되었기 때문에 boardsService 프로퍼티는 BoardsController 클래스 내부에서만 사용 가능합니다.

```
@Controller('boards')
export class BoardsController {
  constructor(private x: string) { }
}

const bar = new BoardsController('Hello World');

console.log(bar);
// Bar { x: 'Hello' }

console.log(bar.x);
//Property 'x' is private and only accessible within class 'BoardsController'.ts(2341)
```